Introduction

- C++ is a superset of C, which has added features to support object-oriented programming.
- C++ supports classes.

C and C++

- C programmers concentrate on writing functions.
- C++ programmers concentrate on creating their own user-defined types called classes.
- Classes in C++ are a natural evolution of the C notion of **`struct`**.

Basic concepts of OOPS

- Objects
- Classes
- Inheritance
- Data Abstraction
- Data Encapsulation
- Polymorphism
- Overloading
- Reusability

Objects:

- Object is the basic unit of object-oriented programming. Objects are identified by its unique name.
- An object represents a particular instance of a class.
- An Object is a collection of data members and associated member functions also known as methods.

Classes:

- Classes are data types based on which objects are created. Objects with similar properties and methods are grouped together to form a Class.
- A Class represent a set of individual objects. Characteristics of an object are represented in a class as **Properties**.
- The actions that can be performed by objects becomes functions of the class and is referred to as **Methods**.

Classes:

- No memory is allocated when a class is created. Memory is allocated only when an object is created, i.e., when an instance of a class is created.

Inheritance:

- Inheritance is the process of forming a new class from an existing class or *base class*.
- The base class is also known as *parent class* or *super class*, The new class that is formed is called *derived class*.
- Derived class is also known as a *child class* or *sub class.* Inheritance helps in reducing the overall code size of the program, which is an important concept in object-oriented programming.

Data Abstraction:

- Data Abstraction increases the power of programming language by creating user defined data types.
- Data Abstraction also represents the needed information in the program without presenting the details.

Data Encapsulation:

- Data Encapsulation combines data and functions into a single unit called Class.
- When using Data Encapsulation, data is not accessed directly; it is only accessible through the functions present inside the class.
- Data Encapsulation enables the important concept of data hiding possible.

Polymorphism:

- Polymorphism allows routines to use variables of different types at different times.
- An operator or function can be given different meanings or functions. Polymorphism refers to a single function or multi-functioning operator performing in different ways.

Overloading:

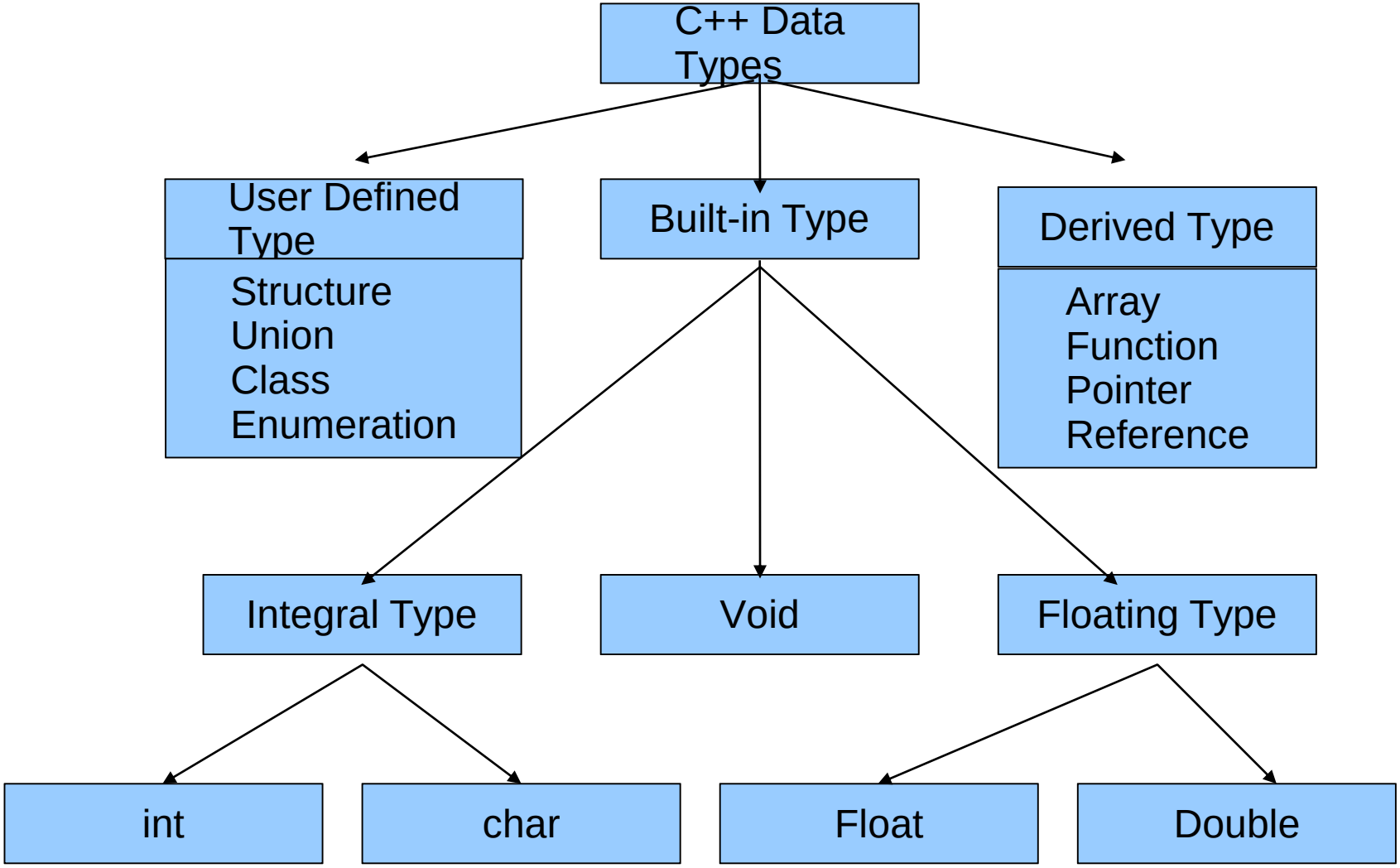
- Overloading is one type of Polymorphism. It allows an object to have different meanings, depending on its context.
- When an exiting operator or function begins to operate on new data type, or class, it is understood to be overloaded.

Reusability:

- This term refers to the ability for multiple programmers to use the same written and debugged existing class of data.
- This is a time saving device and adds code efficiency to the language. Additionally, the programmer can incorporate new features to the existing class, further developing the application and allowing users to achieve increased performance.
- This time saving feature optimizes code, helps in gaining secured applications and facilitates easier maintenance on the application.

- C++ introduces a new kind of variables known as reference variables.
- Provides aliases for previously defined variables.
- **Syntax: Data type & reference_variable= variable;**
- **EX:int total;**
- **int & sum = total;**
- Both variables refer to the same object in the memory.

Basic Data Types



VOID TYPE

- Two normal uses of void:

1)To specify the return type

2)To specify the empty parameter list

- Another interesting use is in the declaration of generic pointers
  -
- A generic pointer can be assigned a pointer value of any basic data type.
  -
  -
- Dereferencing a pointer to a void value is illegal
  -

VOID TYPE

- Assigning a void pointer to a non-void pointer without using a cast to non-void pointer type.This is not allowed in c++
- void *p1;
- char *p2;
- p2 = p1;

- P2 = (char *)p1;

User-Defined Data Types

- **Enumeration Type** :An enumeration type is a user defined type which provides a way for attaching names to numbers,thereby increasing readability of the code.
- The enum keyword automatically enumerates a list of words by assigning them values 0,1,2 and so on.
- This facility provides a alternative means for creating symbolic constants .
- **EX:enum colour {red,blue,green,yellow};**
- This DT differs slightly in c++ when compared with c.
- *The tag names can be used to declare new variables

User-Defined Data Types

Enumeration Type

Ex: colour background;

* C defines the types of enums to be imts. In c++, Each enumerated DT retains its own separate type.

EX: colour background =blue;

Colour background =8;

Colour background =(colour) 8;

- An enumerated value can be used in place of an int value.
- EX: int a= blue;

User-Defined Data Types
Enumeration Type

- C++ allows creation of anonymous enums
- EX: enum{off , on};
- These constants can be referenced in the same manner as regular constants.
- EX: int switch_1 = off;

  » int switch_2 = on;
- Practically ,enumeration is used to define symbolic constants for a switch statement.

Reference variables

- C++ introduces a new kind of variables known as reference variables.
- Provides aliases for previously defined variables.
- **Syntax: Data type & reference_variable= variable;**
- **EX:int total;**
- **int & sum = total;**
- Both variables refer to the same object in the memory.

Reference variables

- A reference variable must be initialized at the time of declaration.
- Initialization of a reference variable is completely different from assignment .
- **EX:**
- **1)int a[10];** **2) int x;**
- **int & x = a[10];** **int *p = &x;**
- **char & y = '\n';** **int & y= *p;**
-
- **3) int & q = 32;**

Reference variables

- A major application of reference variables is in passing arguments to functions.
- **EX: void fun(int & m)**
- **{ m=m++; }**
- **main( )**
- **{ int x=10;**
- **fun(x); }**
- Call by reference is used in oop because it permits the manipulation of objects by reference ,& eliminates the copying of object parameters back and forth.
- References can be created for user defined DT's also

Operators in c++

- C++ has rich set of operators. All c operators are valid in c++ also. In addition ,c++ introduces some new operators .
- **: :** scope resolution operator
- **: :*** pointer to member declarator
- **->*** pointer to member operator
- **.*** ----------,,----------
- **Delete** memory release operator
- **Endl** l line feed operator
- **New** memory allocation operator
- **Setw** field width operator

: : scope resolution operator

- Like C ,C++ is also a block -structured language
- Blocks and scopes can be used in constructing programs.
- The scope of the variable extends from the point of its declaration till the end of the block containing the declaration.
- Blocks in C++ are often nested.
- In c, the global version of variable cannot be accessed from within the inner block.c++ resolves this problem by introducing a new operator **::**
- **Syntax :: variable name**

: : scope resolution operator

```
#include <iostream>
using namespace std;
int a=10;
int main( )
{   int a=20;
        {
            int b=a;
             int a =30;
            cout<<b<<a <<:: a<<endl;
        }
      cout <<a<<:: a<<endl;
      return(0);
}
```

: : scope resolution operator

- A major application of scope resolution operator is in classes to identify class to which a member functions belongs.

Memory Management operators

- In programming there may be scenarios where programmers may not know the memory needed until run time. In this case, the programmer can opt to reserve as much memory as possible
- Assigning the maximum memory space needed to tackle this situation. This would result in wastage of unused memory spaces.
- Memory management operators are used to handle this situation in C++ programming language

Memory Management operators

- There are two types of memory management operators in C++:

  * new
  * delete

- These two memory management operators are used for allocating and freeing memory blocks in efficient and convenient ways.

- **The general syntax :**
- **pointer variable = new datatype;**
- **example:**
- **int *a=new int;**
- Dynamic variables are never initialized by the compiler. Therefore, the programmer should make it a practice to first assign them a value.

- **int *a = new int;**
- ***a = 20;**
- **or**
- **int *a = new int(20);**
- **Syntax:**
- **pt_var =new dT[10]; //for array**

delete operator:

- The delete operator in C++ is used for releasing memory space when the object is no longer needed. Once a new operator is used, it is efficient to use the corresponding delete operator for release of memory.
- **syntax:**
- **delete pointer variable;**

delete [size] pointer variable;

- **Ex :delete p;**
- **delete [ ] p;**

Delete operator

Some of the important points :

- * The programmer must take care not to free or delete a pointer variable that has already been deleted.
- * Overloading of new and delete operator is possible .*The programmer must take care not to free or delete pointer variables that have not been allocated using a new operator.

* Null pointer is returned by the new operator when there is insufficient memory available for allocation.